

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_f39953fd-98d\agent-ad06f8af1e83a7d10.jsonl`
- SHA-256: `17cd63b0bc8a09dcb384010c43d28015fadbc12081feaeabad60547539ca2385`
- Source modified: `2026-06-22T15:05:56+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_f39953fd-98d`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T15:03:44.426Z)

Adversarially verify this finding by reading the cited code in C:/Users/avidu/Projects/bhi-cloudrun (run `cd C:/Users/avidu/Projects/bhi-cloudrun && git diff origin/master...HEAD`). Real defect to fix, or false alarm/nit? Default is_real=false if it doesn't hold up.

Lens: default-off-regression

Severity: nit

Title: Cloud success path returns DB-row shaped play_segments (with extra keys) vs in-process segmenter-shaped dicts ? verified compatible

File: backend/main.py

Detail: The in-process success branch (backend/main.py:638-644) returns play_segments = the segmenter's own dicts (keys id/start_time/end_time/duration/metadata). The cloud success branch (backend/main.py:521-526) returns play_segments = db_instance.query_play_segments(...) rows, which carry the same load-bearing keys (id/start_time/end_time/duration/metadata) PLUS server/receiver/events:[] (backend/infrastructure/database.py:597-610). This is the exact shape /api/query already returns, so the SPA and tests that read play_segments by those key names are unaffected. test_full_api_process_to_query_via_cloud and test_cloud_dispatch_ingests_segments assert the cloud rows round-trip through /api/query correctly, and all 112 tests in the changed region (test_api_endpoints/test_reingest_safety/test_async_jobs/test_job_worker_loop/test_job_worker_module_seam/test_edge_auth/...) still pass. Noted only because the two success branches now produce a superset shape, not an identical one ? but no consumer depends on the segmenter-only shape.

Fix: None needed ? shapes are key-compatible and exercised by tests.

2. assistant (2026-06-22T15:03:49.170Z)

I'll verify this finding by examining the cited code in the bhi-cloudrun repo.

3. user (2026-06-22T15:03:51.947Z)

---HEAD---

304252b feat(serving): wire /api/process to dispatch DETECT to Cloud Run (M6 -> API), default-OFF

---DIFF STAT---

```
backend/main.py | 100 ++++++
tests/test_api_cloud_dispatch.py | 212 ++++++
2 files changed, 312 insertions(+)
```

4. assistant (2026-06-22T15:03:53.883Z)

Let me read the cited code regions.

5. user (2026-06-22T15:03:55.744Z)

```

diff --git a/backend/main.py b/backend/main.py
index 4e5a19a..04ffb67 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -1,10 +1,12 @@
import os
import sys
import argparse
+import csv
import json
import logging
import shutil
import socket
+import tempfile
import threading
import uuid
import webbrowser

@@ -436,6 +438,94 @@ def download_reel(filename: str):
    return FileResponse(path, media_type=media, filename=name)

+def _ingest_remote_segments(video_id: str, outputs_uri: str, storage_config: dict) -> int:
+    """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from the
+    bucket and
+    persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters call ? so
+    ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they were
+    produced.
+    Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst pattern
+    (cloud
+    backends REQUIRE a dst). A malformed row is skipped, never aborts the ingest."""
+    uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
+    ref = storage.parse_uri(uri)
+    dst = os.path.join(tempfile.mkdtemp(prefix="cr-ingest-"), ref.name or "intervals.csv")
+    local = storage.fetch(uri, dst, config=storage_config)
+    n = 0
+    with open(local, newline="", encoding="utf-8") as f:
+        for row in csv.DictReader(f):
+            try:
+                start, end = float(row["start"]), float(row["end"])
+            except (KeyError, TypeError, ValueError):
+                continue
+            meta: Dict[str, Any] = {"source": "cloud_run",
+                                   "ending_reason": (row.get("ending_reason") or "").strip()}
+            sc = (row.get("shot_count") or "").strip()
+            if sc:
+                try:
+                    meta["shot_count"] = int(float(sc))
+                except ValueError:
+                    meta["shot_count"] = sc
+            db_instance.add_play_segment(video_id, start, end, metadata=meta)
+            n += 1
+    return n
+
+def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name: str, val_results,
+                             play_wm: int, cand_wm: int, notify) -> Optional[Dict[str, Any]]:
+    """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
+    ``deployment.compute_target
+    == "cloud_run``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
+    intervals into
+    THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
+
+    Returns a response dict (``success`` | ``failed``) when the cloud path is taken, or
+    ``None`` to
+    fall through to the in-process segmenter ? **DEFAULT-OFF**: ``get_compute_backend`` returns

```

```

``None``
+   for every non-``cloud_run`` target, so the in-process path stays byte-identical.""
+   from backend.compute import ComputeJobSpec, get_compute_backend
+
+   compute = get_compute_backend(global_config)
+   if compute is None:
+       return None # default-OFF ? run the in-process segmenter below (unchanged)
+
+   storage_cfg = getattr(global_config, "storage", None)
+
+   def _failed(msg: str) -> Dict[str, Any]:
+       # Shape-match the in-process segmenter-fail branch + restore prior good rows (none
written yet).
+       db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
+       return {"video_id": video_id, "status": "failed", "message": msg,
+           "results": [r.model_dump() for r in val_results], "play_segments": []}
+
+   # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local to this
box.
+   try:
+       input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
+   except ValueError as e:
+       return _failed(f"cloud-serving: unresolvable input '{req.video_path}': {e}")
+   if input_ref.backend == "local":
+       return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to this
machine "
+
+           "can't be read by the Cloud Run job. Put the video in your bucket
first.")
+   out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
+   if not out_root:
+       return _failed("cloud-serving needs storage.output_root set to a bucket (e.g.
gs://your-bucket) "
+
+           "? that's where the Cloud Run job writes the result.")
+
+   notify("dispatching (cloud_run)")
+   logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
req.video_path, out_root)
+   spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root, segmenter=seg_name)
+   try:
+       result = compute.run_infer(spec) # dispatch the Cloud Run Job + bucket-poll to
completion
+   except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job failure,
not a 500
+       logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
+       return _failed(f"cloud dispatch error: {e}")
+   if result.returncode != 0:
+       return _failed(f"cloud job failed (returncode {result.returncode}).")
+
+   notify("ingesting (cloud_run)")
+   storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else {}
+   _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
+   # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then drops the
old ones.
+   segments = [s for s in db_instance.query_play_segments(video_id, {}) if int(s.get("id", 0))
> play_wm]
+   _finalize_reingest(video_id, segments, play_wm, cand_wm)
+   return {"video_id": video_id, "status": "success",
+       "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
+       "results": [r.model_dump() for r in val_results], "play_segments": segments}
+
+
+   def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
+       """The full hash ? validate ? segment ? report pipeline for one video.

```

```

@@ -508,6 +598,16 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
    # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old only
    # if the run yields segments; otherwise drop the new partial rows and keep the old.
    play_wm, cand_wm = db_instance.segment_watermarks(video_id)
+    # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run, run the
heavy
+    # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
+    # DEFAULT-OFF: returns None for every non-cloud target ? the in-process segmenter below
runs
+    # byte-identically. (player_pool/max_frames aren't threaded to the worker `infer` CLI
yet, so
+    # they're dropped on the cloud path in this slice ? a documented follow-up.)
+    remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm, cand_wm,
notify)
+    if remote is not None:
+        segmentation_ran = True
+        response = remote
+        return response
    segmenter = segmenter_cls(db_instance, global_config)
    try:
        result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)

```

6. assistant (2026-06-22T15:04:03.954Z)

Now let me look at the in-process success branch and the exact line numbers cited in the finding.

7. user (2026-06-22T15:04:04.869Z)

```

600         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
601         # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
602         # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
603         # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
604         # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
605         # they're dropped on the cloud path in this slice ? a documented follow-up.)
606         remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm,
cand_wm, notify)
607         if remote is not None:
608             segmentation_ran = True
609             response = remote
610             return response
611         segmenter = segmenter_cls(db_instance, global_config)
612         try:
613             result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
614         except Exception:
615             # A raising segmenter must not leave half-written rows: restore the pre-run
616             # state (same destroy-after-confirm contract as the Failure branch), then
let
617             # the job worker record the failure.
618             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
619             raise
620         segmentation_ran = True
621
622         if isinstance(result, Failure):
623             logger.error(f"[API] Segmenter failed: {result.message}")
624             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
625             response = {

```

```

626         "video_id": video_id,
627         "status": "failed",
628         "message": f"Segmenter '{seg_name}' failed: {result.message}",
629         "results": [r.model_dump() for r in val_results],
630         "play_segments": [],
631     }
632     return response
633
634     segments = result.value if hasattr(result, "value") else result
635     notify("finalizing")
636     _finalize_reingest(video_id, segments, play_wm, cand_wm)
637
638     response = {
639         "video_id": video_id,
640         "status": "success",
641         "message": f"Successfully indexed {len(segments)} play segments using
642 '{seg_name}' segmenter.",
643         "results": [r.model_dump() for r in val_results],
644         "play_segments": segments,
645     }
646     return response
647 finally:
648     # Always emit the per-video observability report (next to the video, or to
649     # report.output_dir). Never raises. Surface the paths in the response.
650     paths = emit_indexer_report(
651         db=db_instance, video_id=video_id, video_path=req.video_path,
652         config=global_config,
653         val_results=val_results, val_context=val_context,
654         segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
655         was_reingest=was_reingest, segmentation_ran=segmentation_ran,
656     )
657     if paths and isinstance(response, dict):
658         response["reports"] = paths
659
660 @app.post("/api/process", status_code=202)
661 def process_video(req: ProcessRequest, request: Request):
662     """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
663
664     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
665     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
666     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
667     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
668     """
669     # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
670     single-user,
671     # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
672     missing or

```

8. assistant (2026-06-22T15:04:07.644Z)

Now let me confirm the key claim about what `query\_play\_segments` returns vs. what the in-process segmenter returns. The finding cites database.py:597-610. Let me check the actual shape of both.

9. user (2026-06-22T15:04:08.517Z)

```

570     - event_type: string (e.g. smash, foul)
571     """
572     query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
573     params = [video_id]
574
575     if filters.get("duration_min"):
576         query += " AND r.duration >= ?"
577         params.append(filters["duration_min"])

```

```

578
579         if filters.get("server"):
580             query += " AND r.server = ?"
581             params.append(filters["server"])
582
583         if filters.get("receiver"):
584             query += " AND r.receiver = ?"
585             params.append(filters["receiver"])
586
587         if filters.get("event_type"):
588             # Check if there is an event of a specific type inside this segment
589             query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
590             params.append(filters["event_type"])
591
592         segments_list = []
593         with self._connect() as conn:
594             cursor = conn.cursor()
595             rows = cursor.execute(query, params).fetchall()
596
597         for row in rows:
598             r_dict = dict(row)
599             r_id = r_dict["id"]
600             r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
601
602             # Fetch associated events
603             event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
604             r_dict["events"] = []
605             for er in event_rows:
606                 e_dict = dict(er)
607                 e_dict["details"] = json.loads(e_dict["details"] or "{}")
608                 r_dict["events"].append(e_dict)
609
610             segments_list.append(r_dict)
611
612         return segments_list
613
614     def clear_video_data(self, video_id: str):
615         with self._connect() as conn:
616             conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
617             conn.commit()
618
619     # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
620     # Writers follow the repo convention (swallow + logger.error, return bool); the
621     # worker logs loudly on a failed transition so a job can't silently wedge.
622
623     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
624                   player_pool: Optional[List[str]] = None,

```

10. assistant (2026-06-22T15:04:13.558Z)

Now I understand the DB-row shape. `query\_play\_segments` returns `r.\*` (all columns from `play\_segments` table) plus `metadata` (parsed JSON) and `events: []`. Let me verify the actual table schema to confirm which columns exist (server/receiver/etc.) and verify what the in-process segmenter returns.

11. user (2026-06-22T15:04:15.063Z)

```

186:         CREATE TABLE IF NOT EXISTS play_segments (
---SCHEMA---
        CREATE TABLE IF NOT EXISTS play_segments (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            video_id TEXT NOT NULL,
            start_time REAL NOT NULL,

```

```

end_time REAL NOT NULL,
duration REAL NOT NULL,
server TEXT,
receiver TEXT,
metadata TEXT,
FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE

```

12. assistant (2026-06-22T15:04:18.306Z)

The DB row carries: `id, video\_id, start\_time, end\_time, duration, server, receiver, metadata` plus `events: []`. Now let me confirm the in-process segmenter's dict shape ? what keys it actually produces.

13. user (2026-06-22T15:04:19.861Z)

```

backend/infrastructure/database.py:459:     def add_play_segment(self, video_id: str, start_time:
float, end_time: float, server: Optional[str] = None, receiver: Optional[str] = None, metadata:
Optional[Dict[str, Any]] = None) -> Optional[int]:
---add_play_segment---
    def add_play_segment(self, video_id: str, start_time: float, end_time: float, server:
Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]] = None)
-> Optional[int]:
        try:
            with self._connect() as conn:
                cursor = conn.cursor()
                cursor.execute(
                    "INSERT INTO play_segments (video_id, start_time, end_time, duration,
server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
                    (video_id, start_time, end_time, end_time - start_time, server, receiver,
json.dumps(metadata or {}))
                )
                conn.commit()
            return cursor.lastrowid

```

14. assistant (2026-06-22T15:04:22.176Z)

Now let me find what the in-process segmenter returns as `segments` (the `result.value`). This is the key comparison ? what dict shape does the in-process path produce?

15. user (2026-06-22T15:04:23.357Z)

```

backend/pipeline/segmenters/base.py:24:     def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
backend/pipeline/segmenters/gemini.py:509:     def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
backend/pipeline/segmenters/motion.py:20:     def process_video(self, video_id: str, video_path:
str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
backend/pipeline/segmenters/trajectory_hybrid.py:138:     def process_video(self, video_id: str,
video_path: str, # type: ignore[override]
backend/pipeline/segmenters/yolo_hybrid.py:366:     def process_video(self, video_id: str,
video_path: str, player_pool: List[str] = None, max_frames: int = None) -> List[Dict[str, Any]]:
---base process_video / return---

```

16. user (2026-06-22T15:04:25.925Z)

```

def process_video(self, video_id: str, video_path: str, player_pool: List[str] = None,
max_frames: int = None) -> List[Dict[str, Any]]:
    """
    Processes a video to detect play segments using motion heuristics,
    populates SQLite with detected segments, and indexes detailed events.
    """
    if not player_pool:
        player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]

```

```

cap = cv2.VideoCapture(video_path)
if not cap.isOpened():
    logger.error(f"Segmenter could not open video: {video_path}")
    return Failure(message=f"Could not open video: {video_path}", is_recoverable=False)

final_segments = self._detect_motion_segments(cap, max_frames)

segments_data = self._populate_db_with_events(video_id, final_segments, player_pool)

return Success(value=segments_data)

def _detect_motion_segments(self, cap: Any, max_frames: int = None) -> List[Tuple[float,
float]]:
    """Run the pixel-motion heuristic over ``cap`` and return the detected
    (start, end) play segments.

    Reads/sub-samples frames, builds a per-frame motion signal, smooths it,
    thresholds it into raw segments, then merges dead-time gaps and filters
    out segments shorter than ``min_segment_duration``. ``cap`` is released
    before returning. This is the detection half of ``process_video`` ? it
    performs no DB writes and no fabrication.
    """
    fps = cap.get(cv2.CAP_PROP_FPS)
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    duration = total_frames / fps if fps > 0 else 0

    # Sub-sample settings (5 FPS for analysis)
    analysis_fps = 5.0
    frame_interval = max(1, int(fps / analysis_fps))

    idx_cfg = self.config.get("indexing", {})
    motion_threshold = idx_cfg.get("motion_threshold", 0.08)
    min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
    dead_time_merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)

    prev_gray = None
    motion_signals = []
    timestamps = []

    frame_idx = 0
    processed_count = 0
    t_start = time.time()
    # Emit a progress line roughly every 5% of the video so long runs are
    # observable instead of silent (a full 1080p match is ~tens of minutes).
    progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
    next_progress = progress_step
    logger.info(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
                f"(~{duration/60:.1f} min), analyzing every {frame_interval}th frame")
    while True:
        # Skip decoding for intermediate frames using cap.grab() for high performance
        for _ in range(frame_interval - 1):
            if not cap.grab():
                break
            frame_idx += 1

        ret, frame = cap.read()
        if not ret:
            break

        # Process frame
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        # Apply Gaussian Blur to smooth noise
        gray = cv2.GaussianBlur(gray, (21, 21), 0)

```


The in-process path returns `\_populate\_db\_with\_events(...)` . Let me see exactly what dict shape that builds ? this is the segmenter-shaped dict the finding refers to (keys id/start_time/end_time/duration/metadata).

18. user (2026-06-22T15:04:30.946Z)

```
def _populate_db_with_events(self, video_id: str, final_segments: List[Tuple[float, float]],
                             player_pool: List[str]) -> List[Dict[str, Any]]:
    """Persist ``final_segments`` and their fabricated metadata/events to the DB.

    For each detected segment this assigns a random server/receiver, fabricates
    rich tags, then indexes a serve / 0?3 smash / termination event set ? all
    with the stdlib ``random`` module (the legacy demo's fabricated baseline).
    Returns the per-segment dicts handed back to the caller. The ``random``
    call order here is load-bearing for reproducibility and must not change.
    """
    # Populate SQLite DB with segments and metadata events
    segments_data = []
    for start, end in final_segments:
        # Assign Server and Receiver randomly from pool
        server, receiver = random.sample(player_pool, 2)

        # Formulate dynamic rich tags
        dur = end - start
        metadata = {
            "shot_count": random.randint(4, 25),
            "intensity": "high" if dur > 12 else "medium",
            "avg_speed_kmh": round(random.uniform(40, 85), 1)
        }

        segment_id = self.db.add_play_segment(video_id, start, end, server, receiver,
        metadata)

        if segment_id:
            # Add sub-segment events
            # A: Serve Event (always at start)
            serve_foul = random.random() < 0.15 # 15% chance of serve foul
            self.db.add_event(
                segment_id=segment_id,
                timestamp=start + 0.5,
                event_type="serve",
                player=server,
                details={"status": "foul" if serve_foul else "valid", "type": "low_serve" if
random.random() > 0.5 else "high_serve"}
            )

            # B: Smash Events (1 to 3 random smashes per segment)
            smashes_count = random.randint(0, 3)
            for _ in range(smashes_count):
                smash_time = random.uniform(start + 1.5, end - 1.0)
                smasher = random.choice([server, receiver])
                self.db.add_event(
                    segment_id=segment_id,
                    timestamp=round(smash_time, 2),
                    event_type="smash",
                    player=smasher,
                    details={"speed_kmh": round(random.uniform(120, 250), 1)}
                )

            # C: Segment Termination Event (foul/winner)
            foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
            is_foul = random.random() > 0.4
            error_player = random.choice([server, receiver])

            self.db.add_event(
```

```

        segment_id=segment_id,
        timestamp=end - 0.2,
        event_type="foul" if is_foul else "winner",
        player=error_player,
        details={"type": random.choice(foul_types)} if is_foul else {"shot_type":
"drop" if random.random() > 0.5 else "smash"}
    )

    # Store segment representation to return to caller
    segments_data.append({
        "id": segment_id,
        "start_time": start,
        "end_time": end,
        "duration": dur,
        "server": server,
        "receiver": receiver,
        "metadata": metadata
    })

    return segments_data

```

19. assistant (2026-06-22T15:04:38.564Z)

This is the crux. The in-process segmenter (motion) actually returns dicts with keys: `id, start\_time, end\_time, duration, server, receiver, metadata` ? ****including**** `server` and `receiver`. The cloud path's `query\_play\_segments` returns `id, video\_id, start\_time, end\_time, duration, server, receiver, metadata, events`.

So the finding's characterization is slightly off in detail: it claims the in-process shape is "id/start_time/end_time/duration/metadata" and the cloud shape adds "server/receiver/events". But in reality the in-process motion segmenter ALSO returns `server`/`receiver`. The actual differences are:

- Cloud adds: `video\_id` and `events: []`
- In-process motion lacks: `video\_id` and `events`

Let me check the other segmenters to see if any return a *different* shape, and verify whether any consumer actually depends on the exact key set. Let me check what `/api/query` returns and what the trajectory/yolo segmenters return.

20. user (2026-06-22T15:04:41.376Z)

```

=== trajectory_hybrid return shape ===
303:         "start_time": all_candidate_windows[wi]["start"],
332:         seg["start_time"] = float(seg["start_time"]) + w_start
335:         valid.append(seg)
359:     start = float(segment["start_time"])
372:     segments_data.append({
373:         "id": segment_id, "start_time": start, "end_time": end,

=== yolo_hybrid ===
292:         segment["start_time"] = float(segment["start_time"]) + w_start
327:     start = float(segment["start_time"])
354:     segments_data.append({
355:         "id": segment_id,
356:         "start_time": start,

```

21. user (2026-06-22T15:04:45.329Z)

```

=== trajectory_hybrid 372-382 ===
    segments_data.append({
        "id": segment_id, "start_time": start, "end_time": end,
        "duration": dur, "metadata": metadata,
    })
except Exception as e:

```

```

        logger.warning("Skipping invalid segment struct: %s", e)

    return segments_data
=== yolo_hybrid 354,368 ===
        segments_data.append({
            "id": segment_id,
            "start_time": start,
            "end_time": end,
            "duration": dur,
            "metadata": metadata
        })
    except Exception as e:
        logger.warning(f"Skipping invalid segment struct: {e}")

    return segments_data

def process_video(self, video_id: str, video_path: str, player_pool: List[str] = None,
max_frames: int = None) -> List[Dict[str, Any]]:
    self.person.lazy_load_model()

```

22. assistant (2026-06-22T15:04:51.107Z)

This is important context. The in-process segmenters already produce **inconsistent** shapes among themselves:

```

- `motion`: `id, start_time, end_time, duration, server, receiver, metadata`
- `trajectory_hybrid` / `yolo_hybrid`: `id, start_time, end_time, duration, metadata` (no
server/receiver)

```

So the codebase already tolerates variable `play\_segments` shapes across segmenters even on the in-process path. The finding's "exact shape" framing is imprecise, but its core conclusion (consumers only depend on the load-bearing keys `id/start\_time/end\_time/duration/metadata`, which all paths share) is supported.

Now let me verify the consumer claim: do the SPA/tests/`api/compile` read `play\_segments` only by those shared key names? Let me check the cited tests and the compile/query consumers.

23. user (2026-06-22T15:04:52.714Z)

```

=== tests referenced in finding ===
105: def test_cloud_dispatch_ingests_segments(cloud_env):
191: def test_full_api_process_to_query_via_cloud(cloud_env):

```

24. user (2026-06-22T15:04:55.678Z)

```

105     def test_cloud_dispatch_ingests_segments(cloud_env):
106         db, _bucket, monkeypatch = cloud_env
107         fake = _FakeCloudClient()
108         _inject_fake(monkeypatch, fake)
109
110         out = m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid"))
111
112         assert out["status"] == "success"
113         assert len(out["play_segments"]) == 2
114         rows = db.query_play_segments(out["video_id"], {})
115         assert len(rows) == 2
116         metas = [_meta(r) for r in rows]
117         assert all(md["source"] == "cloud_run" for md in metas)
118         assert {md.get("shot_count") for md in metas} == {5, 3}
119         assert {md.get("ending_reason") for md in metas} == {"winner", "unforced_error"}
120
121
122     def test_cloud_dispatch_builds_correct_spec(cloud_env):
123         _db, bucket, monkeypatch = cloud_env

```

```

124         fake = _FakeCloudClient()
125         _inject_fake(monkeypatch, fake)
126
127         m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
128         segmenter="wasb_hybrid"))
129
130         assert len(fake.calls) == 1 # exactly one Cloud Run Job execution
131         argv = fake.calls[0]
132         assert argv[argv.index("--video-uri") + 1] == "gs://b/clip.mp4"
133         assert argv[argv.index("--out-uri") + 1] == str(bucket)
134         assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
135         joined = " ".join(argv)
136         assert "deployment.compute_target=local_gpu" in joined # container runs INLINE
137         (never re-dispatch)
138         assert "indexing.detector_impl=native" in joined
139
140     def test_cloud_failure_marks_failed_no_rows(cloud_env):
141         db, _bucket, monkeypatch = cloud_env
142         # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-
143         confirm).
144         db.add_video("vidLocalPrior", "gs://b/clip.mp4", "processed", [])
145         fake = _FakeCloudClient(returncode=1)
146         _inject_fake(monkeypatch, fake)
147
148         out = m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
149         segmenter="wasb_hybrid"))
150         assert out["status"] == "failed"
151         assert "returncode 1" in out["message"]
152         assert out["play_segments"] == []
153         assert db.query_play_segments(out["video_id"], {}) == []
154
155     def test_cloud_requires_a_bucket_input(cloud_env, tmp_path):
156         _db, _bucket, monkeypatch = cloud_env
157         fake = _FakeCloudClient()
158         _inject_fake(monkeypatch, fake)
159         local_path = str(tmp_path / "local_only.mp4")
160
161         out = m._execute_processing(m.ProcessRequest(video_path=local_path,
162         segmenter="wasb_hybrid"))
163         assert out["status"] == "failed"
164         assert "cloud input" in out["message"]
165         assert fake.calls == [] # never dispatched a local-only path
166
167     def test_default_off_runs_in_process(tmp_path, monkeypatch):
168         """compute_target='' ? get_compute_backend returns None ? the in-process segmenter
169         runs
170         byte-identically (the gate is inert). No Cloud Run dispatch."""
171         db = Database(str(tmp_path / "t.db"))
172         cfg = AppConfig() # compute_target defaults to ""
173         monkeypatch.setattr(m, "db_instance", db)
174         monkeypatch.setattr(m, "global_config", cfg)
175         monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(),
176         {}))
177
178         video = tmp_path / "v.mp4"
179         video.write_bytes(b"hashable-bytes")
180
181         class _InProcSeg:
182             def __init__(self, db_, cfg_):
183                 self.db = db_
184
185             def process_video(self, vid, path, *a, **k):
186                 sid = self.db.add_play_segment(vid, 0.0, 5.0)

```

```

182         return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0,
"metadata": {}}]
183
184     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
185     out = m._execute_processing(m.ProcessRequest(video_path=str(video),
segmenter="motion"))
186     assert out["status"] == "success"
187     assert len(out["play_segments"]) == 1
188     assert "cloud_run" not in out["message"]
189
190
191     def test_full_api_process_to_query_via_cloud(cloud_env):
192         """End-to-end through the public API: POST /api/process (gs:// input) ? the inline
worker dispatches
193         to (fake) Cloud Run ? job done ? POST /api/query returns the ingested segments."""
194         db, _bucket, monkeypatch = cloud_env
195         fake = _FakeCloudClient(video_id="vidCloud2")
196         _inject_fake(monkeypatch, fake)
197         client = TestClient(m.app)
198
199         r = client.post("/api/process", json={"video_path": "gs://b/clip.mp4", "segmenter":
"wasb_hybrid"})
200         assert r.status_code == 202
201         job_id = r.json()["job_id"]
202
203         job = client.get(f"/api/jobs/{job_id}").json()
204         assert job["status"] == "done"
205         video_id = job["video_id"]
206         assert video_id
207
208         q = client.post("/api/query", json={"video_id": video_id})
209         assert q.status_code == 200
210         segs = q.json()["play_segments"]
211         assert len(segs) == 2
212         assert all(_meta(s)["source"] == "cloud_run" for s in segs)
213

```

25. assistant (2026-06-22T15:04:59.444Z)

The tests confirm consumers read `play\_segments` only by the shared key names (`metadata`, `length`, `source`). Now let me run the full test suite for the changed region to confirm the finding's claim that all tests pass, and verify there's no consumer that depends on the segmenter-only shape (e.g., compile reading `server`/`receiver` or iterating exact keys).

26. user (2026-06-22T15:05:01.188Z)

```

=== Does /api/compile or any consumer read server/receiver/events keys off play_segments? ===
492:         "results": [r.model_dump() for r in val_results], "play_segments": []}
522:     segments = [s for s in db_instance.query_play_segments(video_id, {}) if int(s.get("id",
0)) > play_wm]
526:         "results": [r.model_dump() for r in val_results], "play_segments": segments}
591:         "play_segments": [],
630:         "play_segments": [],
643:         "play_segments": segments,
795: def query_play_segments(req: QueryRequest):
802:     segments = db_instance.query_play_segments(req.video_id, filters)
803:     return {"play_segments": segments}
812:     all_segments = db_instance.query_play_segments(req.video_id, {})

```

27. user (2026-06-22T15:05:03.575Z)

```

805     @app.post("/api/compile")
806     def compile_highlights(req: CompileRequest):
807         video = db_instance.get_video(req.video_id)

```

```

808         if not video:
809             raise HTTPException(status_code=404, detail="Indexed video record not found.")
810
811         # Retrieve matching play segments from db
812         all_segments = db_instance.query_play_segments(req.video_id, {})
813         matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
814
815         if not matched_segments:
816             raise HTTPException(status_code=400, detail="No matching play segments found to
817 stitch.")
818
819         # Reject path traversal / absolute output names (os.path.join would otherwise let an
820 # absolute or '..'-laden output_filename write anywhere on disk).
821         try:
822             out_name = safe_output_filename(req.output_filename)
823         except ValueError as e:
824             raise HTTPException(status_code=400, detail=str(e)) from e
825
826         # The configured shot-count floor (stitching.min_shot_count) applies on the API
827 # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
828 shot_count
829 # metadata are exempt: the floor filters known counts only, so explicitly
830 # requested manual/legacy segments can't silently vanish under the default floor.
831 stitching = getattr(global_config, "stitching", None) or StitchingConfig()
832 kept = []
833 for s in matched_segments:
834     meta = s.get("metadata") or {}
835     sc = meta.get("shot_count") if isinstance(meta, dict) else None
836     try:
837         if sc is not None and int(sc) < stitching.min_shot_count:
838             continue
839     except (TypeError, ValueError):
840         pass # unparseable count -> treat as unknown, keep
841     kept.append(s)
842 if len(kept) < len(matched_segments):
843     logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
844 dropped "
845             f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
846 segments below the floor.")
847 if not kept:
848     raise HTTPException(status_code=400,
849             detail=f"All {len(matched_segments)} matching segments fall
850 below "
851             f"stitching.min_shot_count={stitching.min_shot_count}.")
852
853 # Extract start/end time pairs
854 time_pairs = [(s["start_time"], s["end_time"]) for s in kept]

```

28. assistant (2026-06-22T15:05:07.914Z)

Critically, ``/api/compile`` itself reads segments from ``db_instance.query_play_segments(...)`` ? i.e. it always works off the ****DB-row shape****, never off the segmenter-returned ``play_segments`` dict in the response. The ``play_segments`` key in the ``/api/process`` response is purely informational/for the SPA. Compile re-queries the DB. This fully confirms there is no consumer dependence on the segmenter-only response shape.

Now let me run the cited test region to verify the "all 112 tests pass" claim.

29. user (2026-06-22T15:05:25.168Z)

```

..... [100%]
64 passed in 13.58s

```

30. assistant (2026-06-22T15:05:28.205Z)

All 64 in that subset pass. Let me run the rest of the cited test files to confirm the broader "112 tests" claim and that nothing in the changed region breaks.

31. user (2026-06-22T15:05:34.058Z)

```
..... [100%]  
41 passed in 3.61s
```

32. assistant (2026-06-22T15:05:45.927Z)

All cited tests pass (64 + 41 = 105; the finding's "112" likely includes a couple more files, but the region is green). The verification is complete.

Verdict

I've adversarially verified the finding by reading all the cited code and running the tests.

****Core facts confirmed:****

- Cloud success branch (`backend/main.py:521-526`) returns `play\_segments = db\_instance.query\_play\_segments(...)` ? DB-row shape: `id, video\_id, start\_time, end\_time, duration, server, receiver, metadata, events[:]` (table schema at database.py:186; query at database.py:597-610).
- In-process success branch (`backend/main.py:638-644`) returns `play\_segments = segments` = the segmenter's own dicts.
- The shared load-bearing keys (`id/start\_time/end\_time/duration/metadata`) are present on both paths, and the cloud shape is a superset.

****No consumer depends on the segmenter-only shape**** ? strongest evidence: `/api/compile` (main.py:805-849) re-queries the DB itself (`query\_play\_segments`), so it always operates on the DB-row shape regardless of what the `/api/process` response carried; the response `play\_segments` is purely informational. The cited tests read only by shared key names and all pass (verified locally: 105 of the cited tests green, 0 failures).

****One inaccuracy in the finding's framing**** (does not change the conclusion): the finding asserts the in-process shape is exactly `id/start\_time/end\_time/duration/metadata` and that the cloud path *adds* `server/receiver/events`. In reality:

- The `motion` segmenter already returns `server`/`receiver` too (motion.py `populate\_db\_with\_events`), while `trajectory\_hybrid`/`yolo\_hybrid` do ****not**** (they return only `id/start\_time/end\_time/duration/metadata`). So in-process shapes already vary across segmenters today.
- The true cloud-vs-in-process deltas are `video\_id` and `events[:]` (present on cloud, absent in-process), not `server/receiver`.

This actually *strengthens* the finding's conclusion: the codebase already tolerates heterogeneous `play\_segments` response shapes across segmenters, so a superset cloud shape is a non-issue. It remains a nit (observation, no fix needed), exactly as labeled.

33. user (2026-06-22T15:05:52.805Z)

Structured output provided successfully

34. assistant (2026-06-22T15:05:55.986Z)

Verified. The finding is an accurate observation but not a real defect ? it self-classifies correctly as a no-fix-needed nit, and my verification confirms shapes are key-compatible with no consumer depending on the segmenter-only shape (notably `/api/compile` re-queries the DB rather than reading the response dict). All cited tests pass locally (105 green, 0 failures). I also noted one factual imprecision in the finding's framing that does not change the verdict.